

C++ 算法手的 Java 速通

从 STL、裸指针到 Collections、引用与 JVM

基于 47 份真实 Leet C++ 解法的迁移手册

SZY

2026 年 7 月 13 日

写给已经会 C++ 的你

你不需要再学一遍算法。你的题库已经包含 3Sum、课程表、二叉树最大路径和、硬币兑换、LRU、数据流中位数、图克隆、时间键值存储和 FHQ Treap：这说明你会的是建模、复杂度和数据结构。

真正需要迁移的是**默认动作**：C++ 里你自然会写 `vector`、`unordered_map`、`priority_queue`、裸指针和 `new`；Java 里应该自然写 `ArrayList`、`HashMap`、`PriorityQueue`、对象引用和 GC。算法几乎不变，容器 API、类型系统、对象语义和性能边界变了。

本书的目标不是介绍 Java 全家桶，而是让你在一周内能够：

1. 在 LeetCode 用 Java 不查语法地写出你已经会的题；
2. 理解 Java 的引用、泛型、Comparator、集合复杂度和整数坑；
3. 将 C++ 的手写链表、堆、图搜索和 DP 迁移成可读、可维护的 Java；
4. 知道哪些 C++ 习惯应保留，哪些在 Java 中反而会制造 bug。

目录

写给已经会 C++ 的你	i
I 先建立正确心智模型	1
1 七天迁移路线：算法不变，默认动作变	2
1.1 先盘点你的现有能力	2
1.2 一周安排	2
1.3 先装什么	3
1.4 从 C++ 调试习惯切换	3
2 语言迁移：从值、指针和模板到引用、对象和泛型	4
2.1 值、引用与 null	4
2.2 数组优先于 ArrayList 的场景	4
2.3 类、内部类和节点	5
2.4 泛型的两条规则	5
II 把你的算法习惯搬过去	6
3 STL 到 Collections：把容器选择变成条件反射	7
3.1 核心映射表	7
3.2 哈希表：从 operator[] 到显式默认值	7
3.3 队列与双端队列	8
3.4 双堆：默认方向和 C++ 反过来	8
3.5 排序与 Comparator	9
4 裸指针、节点和内存：Java 中该怎么想	10
4.1 指针不等于引用，但节点算法仍然一样	10
4.2 LRU：先会手写，再知道标准库	11
4.3 GC 不是“不需要管理内存”	11
4.4 你该保留的 C++ 不变量意识	12

5 把你的题逐类改写：图、树、DP 与二分	13
5.1 课程表：从手写邻接链表到 ArrayList	13
5.2 二叉树最大路径和：返回值与全局答案	14
5.3 硬币兑换：数组、哨兵和循环方向	14
5.4 二分：不要照搬 iterator，照搬不变量	15
5.5 字符串：StringBuilder 是你的默认工具	15
 III 刷题落地	 16
6 Java 刷题 workflow：写得快，也要写得稳	17
6.1 LeetCode 模板	17
6.2 从你代码库抽出的题型清单	17
6.3 提交前 90 秒检查	18
6.4 性能不降级的三个动作	18
 7 速查表与最后冲刺	 19
7.1 最常用 Java 片段	19
7.2 C++ 到 Java 的高频替换	20
7.3 你应该保留的 C++ 优点	20
 A Leet C++ 源码全量索引	 21
 B 你的 Java 迁移题单	 23
B.1 第一组：数组、双指针、排序与基础 API	23
B.2 第二组：对象引用、链表与设计题	23
B.3 第三组：字符串、栈和构造	24
B.4 第四组：DP、树、图、堆与最短路	24

Part I

先建立正确心智模型

Chapter 1

七天迁移路线：算法不变，默认动作变

1.1 先盘点你的现有能力

你的代码库不是“Hello World”级别：`course-schedule.cxx` 用拓扑排序，`find-median-from-data-stream` 用双堆，`time-based-key-value-store.cxx` 用哈希表加二分，`lru-cache.cxx` 手写双向链表，`fhq-treap.cxx` 维护随机平衡树。这些核心思想在 Java 中全部成立。

已有 C++ 能力	Java 中保持不变	需要重新形成肌肉记忆的部分
复杂度分析	$O(n)$ 、 $O(n \log n)$ 不变	装箱、对象分配、GC 影响常数
DFS/BFS/DP	状态设计和递推不变	<code>ArrayDeque</code> 、二维数组、递归深度
STL 选型	哈希、堆、排序思想不变	<code>Collections API</code> 与 <code>Comparator</code>
节点/图建模	邻接表和引用关系不变	没有析构；引用可为 <code>null</code>
手写数据结构	不变量不变	优先选标准库，少管理生命周期

最重要的迁移原则

不要逐行翻译 C++。先写出算法的**数据流与不变量**，再选择 Java 容器。逐行翻译最容易把 `new/delete`、指针算术、宏和 STL iterator 的偶然形式一起带过去。

1.2 一周安排

Day 1: 语法与工具链：类、引用、数组、泛型、编译运行；把 3Sum 改写成 Java。

Day 2: Collections：把 `vector`、`unordered_map`、`set`、`queue`、`priority_queue` 全部过一遍。

Day 3: 对象与节点：改写反转链表、图克隆、LRU；理解对象引用不是 C++ 原始指针。

Day 4: 图与树：改写课程表、岛屿数量、二叉树最大路径和。

Day 5: DP 与字符串：改写硬币兑换、LCS、编辑距离、正则匹配。

Day 6: 堆、二分和排序：改写数据流中位数、时间键值存储、合并 K 链表。

Day 7: 限时模拟：任选三题，强制只写 Java，复盘 API 卡点。

1.3 先装什么

```

1 // Main.java: 本地最小运行方式
2 public class Main {
3     public static void main(String[] args) {
4         System.out.println(new Solution().solve());
5     }
6 }

```

刷题环境通常只要求一个 `class Solution`。本地建议用 JDK 21 或更高版本，IDE 用 IntelliJ IDEA。你会想念 `bits/stdc++.h`；Java 的正确替代不是一份万能头文件，而是一组稳定的 imports：

```

1 import java.util.*;
2 import java.util.function.*;

```

不要机械追求“零 GC”

LeetCode Java 的首要优化顺序是：算法复杂度、错误的容器选择、字符串拼接、重复排序，然后才是对象数量。为了避免几个对象而把代码写成 C 风格数组和全局状态，通常得不偿失。

1.4 从 C++ 调试习惯切换

问题	C++ 常用动作	Java 常用动作
看容器内容	<code>cout << v[i]</code>	<code>System.out.println(list)</code>
断言不变量	<code>assert(x)</code>	<code>assert x</code> ; (运行时需开启 <code>-ea</code>)
快速哈希	<code>unordered_map</code>	<code>HashMap</code>
64 位整数	<code>long long</code>	<code>long</code>
无穷大	<code>INT_MAX / 3</code>	<code>Integer.MAX_VALUE / 3</code>

立即练习

不看答案，把 `best-time-to-buy-and-sell-stock.cxx` 改成 Java。要求：只用一个 `int minPrice` 和一个 `int best`；这题的目的不是算法，而是让“数组遍历 + `Math.max/min`”成为自然动作。

Chapter 2

语言迁移：从值、指针和模板到引用、对象和泛型

2.1 值、引用与 null

C++ 中你会区分值、引用和指针；Java 只有**基本类型的值**和**对象引用的值**。对象引用可以为 null，但不能做指针算术，也没有显式 delete。

```
1 void append(vector<int>& v, int x) {
2     v.push_back(x);           // 修改调用者的 vector
3 }
```

```
1 void append(List<Integer> list, int x) {
2     list.add(x);              // 引用按值传递，但指向同一个对象
3 }
```

“引用按值传递”是 Java 初学者最容易混淆的点：你可以修改 list 指向的对象，不能通过给参数重新赋值来替换调用者变量。

```
1 void wrong(List<Integer> list) {
2     list = new ArrayList<>(); // 只改了局部变量
3     list.add(42);
4 }
```

2.2 数组优先于 ArrayList 的场景

你的 coin-change.cxx 使用 `vector<int> dp(amount + 1, INF)`。Java 直接用原始数组更快、更少装箱：

```
1 int[] dp = new int[amount + 1];
2 Arrays.fill(dp, INF);
3 dp[0] = 0;
```

当大小动态变化或需要库算法时，再选 `ArrayList<Integer>`。算法题中，`int[]`、`int[][]`、`char[]` 常常优于 `List<Integer>`。

C++ 的 `vector<int>` 不总是 Java 的 `ArrayList<Integer>`

固定长度 DP、邻接矩阵、计数数组：用 `int[]`。需要 `push\_back` 的动态结果、邻接表外层：用 `ArrayList<Integer>`。后者会发生 `int` 到 `Integer` 的装箱。

2.3 类、内部类和节点

你的链表、树、图节点在 C++ 中通常带 `Node*`。Java 中使用类字段：

```
1 class ListNode {
2     int val;
3     ListNode next;
4     ListNode(int val) { this.val = val; }
5 }
6
7 class TreeNode {
8     int val;
9     TreeNode left, right;
10    TreeNode(int val) { this.val = val; }
11 }
```

不要因为习惯裸指针而给每个节点加手动释放逻辑；没有外部引用的对象会由 GC 回收。真正需要警惕的是不必要地保留引用，例如把每次 DFS 的节点都塞进全局 List。

2.4 泛型的两条规则

1. Java 泛型不能直接放基本类型：写 `List<Integer>`，不能写 `List<int>`。
2. 泛型没有 C++ 模板的编译期特化能力：不要试图把 Java 泛型当成 template 元编程。

```
1 Map<String, List<Integer>> graph = new HashMap<>();
2 graph.computeIfAbsent("A", ignored -> new ArrayList<>()).add(3);
```

整数溢出

C++ 里你可能会随手写 `long long`；Java 中务必主动写 `long`。特别是面积、路径和、乘法和比较器：

```
1 long area = (long) width * height;
2 Arrays.sort(points, (a, b) -> Integer.compare(a[0], b[0]));
```

不要写 `a[0] - b[0]` 当 comparator：溢出会破坏排序器传递性。

Part II

把你的算法习惯搬过去

Chapter 3

STL 到 Collections: 把容器选择变成条件反射

3.1 核心映射表

C++	Java	使用建议
<code>vector<int></code>	<code>int[]</code> / <code>ArrayList<Integer></code>	固定数值状态优先数组
<code>unordered_map<K,V></code>	<code>HashMap<K,V></code>	平均 $O(1)$ 查找
<code>unordered_set<T></code>	<code>HashSet<T></code>	去重、访问标记
<code>map<K,V></code>	<code>TreeMap<K,V></code>	有序键、floor/ceiling 查询
<code>set<T></code>	<code>TreeSet<T></code>	有序去重
<code>queue<T></code>	<code>ArrayDeque<T></code>	BFS; 不要用 <code>LinkedList</code> 当默认队列
<code>stack<T></code>	<code>ArrayDeque<T></code>	<code>Stack</code> 是遗留类
<code>priority_queue<T></code>	<code>PriorityQueue<T></code>	默认最小堆, 方向和 C++ 相反
<code>sort(begin,end)</code>	<code>Arrays.sort</code> / <code>List.sort</code>	Comparator 用 <code>compare</code>

3.2 哈希表: 从 `operator[]` 到显式默认值

你的 `TimeMap` 自然写出 `store[key].emplace_back(...)`。Java 没有会自动插入的方括号访问, 最接近、也最清晰的写法是:

```
1 class TimeMap {
2     private final Map<String, List<Entry>> store = new HashMap<>();
3
4     public void set(String key, String value, int timestamp) {
5         store.computeIfAbsent(key, ignored -> new ArrayList<>())
6             .add(new Entry(timestamp, value));
7     }
8
9     public String get(String key, int timestamp) {
10         List<Entry> values = store.get(key);
11         if (values == null) return "";
```

```

12
13     int left = 0, right = values.size() - 1, answer = -1;
14     while (left <= right) {
15         int mid = left + (right - left) / 2;
16         if (values.get(mid).time <= timestamp) {
17             answer = mid;
18             left = mid + 1;
19         } else {
20             right = mid - 1;
21         }
22     }
23     return answer == -1 ? "" : values.get(answer).value;
24 }
25
26 private record Entry(int time, String value) {}
27 }

```

不要执着翻译 upper_bound

Java 的 `Collections.binarySearch` 返回值规则对“最后一个不大于 target”不直观。面试中手写这 10 行二分更不容易错，也能直接表达你在 C++ 代码中的意图。

3.3 队列与双端队列

```

1 Deque<Integer> queue = new ArrayDeque<>();
2 queue.offer(0);           // 入队
3 int course = queue.poll(); // 出队; 空时返回 null
4
5 Deque<Character> stack = new ArrayDeque<>();
6 stack.push('(');
7 char top = stack.pop();

```

Java 的 `ArrayDeque` 不允许 `null`。这反而是好事：如果你想把 `null` 当哨兵，应改用显式状态，而不是混进容器。

3.4 双堆：默认方向和 C++ 反过来

你在 `find-median-from-data-stream.cxx` 中把 `lo` 写成大顶堆、`hi` 写成小顶堆。Java 的 `PriorityQueue` 默认是小顶堆：

```

1 class MedianFinder {
2     private final PriorityQueue<Integer> lo =
3         new PriorityQueue<>(Comparator.reverseOrder()); // 大顶堆
4     private final PriorityQueue<Integer> hi =

```

```
5         new PriorityQueue<>(); // 小顶堆
6
7     public void addNum(int num) {
8         lo.offer(num);
9         hi.offer(lo.poll());
10        if (hi.size() > lo.size()) lo.offer(hi.poll());
11    }
12
13    public double findMedian() {
14        return lo.size() > hi.size()
15            ? lo.peek()
16            : ((long) lo.peek() + hi.peek()) / 2.0;
17    }
18 }
```

中位数的 int 溢出

`(lo.peek() + hi.peek()) / 2.0` 在两个接近 `Integer.MAX_VALUE` 的数相加时会先溢出。先转 `long`，再除。

3.5 排序与 Comparator

```
1 int[][] intervals = {{3, 4}, {1, 2}};
2 Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
3
4 List<String> words = new ArrayList<>(List.of("bb", "a", "ccc"));
5 words.sort(Comparator.comparingInt(String::length));
```

立即练习

把 `time-based-key-value-store.cxx` 和 `find-median-from-data-stream.cxx` 改写为 Java。要求：不使用任何第三方库；给中位数代码加上整数溢出保护。

Chapter 4

裸指针、节点和内存：Java 中该怎么想

4.1 指针不等于引用，但节点算法仍然一样

你的 `clone-graph.cxx` 用 `Node*` 和 `unordered_map<int, Node*>`。Java 中节点变量本质上是对象引用，图克隆的算法没有变化：

```
1 class Node {
2     int val;
3     List<Node> neighbors = new ArrayList<>();
4     Node(int val) { this.val = val; }
5 }
6
7 Node cloneGraph(Node start) {
8     if (start == null) return null;
9
10    Map<Node, Node> copies = new HashMap<>();
11    Deque<Node> queue = new ArrayDeque<>();
12    copies.put(start, new Node(start.val));
13    queue.offer(start);
14
15    while (!queue.isEmpty()) {
16        Node current = queue.poll();
17        for (Node next : current.neighbors) {
18            if (!copies.containsKey(next)) {
19                copies.put(next, new Node(next.val));
20                queue.offer(next);
21            }
22            copies.get(current).neighbors.add(copies.get(next));
23        }
24    }
25    return copies.get(start);
26 }
```


用节点对象当 key

你的原实现用 `node->val` 当哈希 key，隐含“节点值唯一”的前提。Java 直接用 `Map<Node, Node>`，把“节点身份”表达得更准确；这也是面试中更稳健的写法。

4.2 LRU: 先会手写，再知道标准库

你在 `lru-cache.cxx` 维护 dummy head/tail、prev/next 和 `unordered_map<int, Node*>`。这对于理解不变量非常好：哈希表定位节点，双向链表维护最近使用顺序。

生产 Java 的默认选择是 `LinkedHashMap`：

```
1 class LRUCache extends LinkedHashMap<Integer, Integer> {
2     private final int capacity;
3
4     LRUCache(int capacity) {
5         super(capacity, 0.75f, true); // true: access-order
6         this.capacity = capacity;
7     }
8
9     public int get(int key) {
10         return getOrDefault(key, -1);
11     }
12
13     public void put(int key, int value) {
14         super.put(key, value);
15     }
16
17     @Override
18     protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
19         return size() > capacity;
20     }
21 }
```

但 LeetCode 的 `LRUCache` 常要求你展示 $O(1)$ 设计。此时手写节点也完全合理，只需把 C++ 的 `Node*` 改成 `Node`，删除 `delete`，并让 `HashMap<Integer, Node>` 持有节点。

4.3 GC 不是“不需要管理内存”

Java 会回收不可达对象，但不会自动修复逻辑引用。以下情况仍会造成长期内存占用：

- 全局 `static List` 持有每次测试数据；
- LRU 淘汰时只删链表、不删 `HashMap`；
- 图遍历的 `visited` 被复用而没有清空；

- Lambda/内部类无意捕获了大对象。

不要在 Java 里模拟 delete

把节点从链表、Map、队列等所有可达路径断开就够了。调用 `System.gc()` 既不能保证回收，也不是算法题答案。

4.4 你该保留的 C++ 不变量意识

FHQ Treap 的 `split/merge/pushup` 展示了优秀的 C++ 习惯：每个变换后恢复子树大小不变量。Java 写复杂树时仍应保留这种纪律：

```
1 private void pull(Node node) {  
2     if (node != null) {  
3         node.size = size(node.left) + size(node.right) + node.count;  
4     }  
5 }
```

语言换了，不变量、边界条件和析构前断链的意识不该换。

Chapter 5

把你的题逐类改写：图、树、DP 与二分

5.1 课程表：从手写邻接链表到 ArrayList

你的 `course-schedule.cxx` 用数组 `edge[3000]` 加链表节点，完全正确，但需要手动分配节点。Java 的邻接表直接表达为“每个课程对应一个动态列表”：

```
1 boolean canFinish(int numCourses, int[][] prerequisites) {
2     List<List<Integer>> graph = new ArrayList<>();
3     for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
4
5     int[] indegree = new int[numCourses];
6     for (int[] pair : prerequisites) {
7         int course = pair[0], prerequisite = pair[1];
8         graph.get(prerequisite).add(course);
9         indegree[course]++;
10    }
11
12    Deque<Integer> queue = new ArrayDeque<>();
13    for (int course = 0; course < numCourses; course++) {
14        if (indegree[course] == 0) queue.offer(course);
15    }
16
17    int finished = 0;
18    while (!queue.isEmpty()) {
19        int course = queue.poll();
20        finished++;
21        for (int next : graph.get(course)) {
22            if (--indegree[next] == 0) queue.offer(next);
23        }
24    }
25    return finished == numCourses;
26 }
```

你原来的算法一行都没变

初始化入度、入队零入度节点、弹出节点后减少后继入度、统计处理数量：这四步原样保留。变的是“Node* 链表”换成“List<Integer>”。

5.2 二叉树最大路径和：返回值与全局答案

你的 C++ 写法将 `ans` 以引用传入 DFS。Java 常用一个成员变量，或者返回一个 `record`；前者最适合刷题：

```
1 class Solution {
2     private int answer = Integer.MIN_VALUE;
3
4     public int maxPathSum(TreeNode root) {
5         gain(root);
6         return answer;
7     }
8
9     private int gain(TreeNode node) {
10        if (node == null) return 0;
11
12        int left = Math.max(0, gain(node.left));
13        int right = Math.max(0, gain(node.right));
14        answer = Math.max(answer, node.val + left + right);
15        return node.val + Math.max(left, right);
16    }
17 }
```

这里可以比原代码更简洁：负贡献在递归返回后立即归零，因此不需要用 `INT_MIN` 当空节点返回值。

5.3 硬币兑换：数组、哨兵和循环方向

```
1 int coinChange(int[] coins, int amount) {
2     int inf = Integer.MAX_VALUE / 3;
3     int[] dp = new int[amount + 1];
4     Arrays.fill(dp, inf);
5     dp[0] = 0;
6
7     for (int coin : coins) {
8         for (int value = coin; value <= amount; value++) {
9             dp[value] = Math.min(dp[value], dp[value - coin] + 1);
10        }
11    }
```

```
12     return dp[amount] == inf ? -1 : dp[amount];
13 }
```

循环方向不是语法问题

你的 C++ 已经写对：金额从小到大遍历，意味着同一种硬币可重复使用（完全背包）。Java 改写时不要因为换语言而反转循环；循环方向编码的是题意。

5.4 二分：不要照搬 *iterator*，照搬不变量

`find-first-and-last-position-of-element-in-sorted-array.cxx` 和 `TimeMap` 的核心是二分边界。Java 里最可读的模板：

```
1 int firstAtLeast(int[] nums, int target) {
2     int left = 0, right = nums.length; // [left, right)
3     while (left < right) {
4         int mid = left + (right - left) / 2;
5         if (nums[mid] < target) left = mid + 1;
6         else right = mid;
7     }
8     return left;
9 }
```

要找最后一个不大于 `target` 的元素，计算 `firstAtLeast(nums, target + 1) - 1`，但要注意 `target == Integer.MAX\_VALUE` 时不应直接加一。

5.5 字符串：StringBuilder 是你的默认工具

你题库里的 Z 字形变换、整数转罗马、文本两端对齐、最短回文都涉及构造字符串。Java 的 `String` 不可变：

```
1 StringBuilder builder = new StringBuilder();
2 for (char ch : chars) {
3     builder.append(ch);
4 }
5 return builder.toString();
```

循环里的 `answer += ch` 会持续创建新 `String`，复杂度可能从线性退化。C++ 的 `string::push\_back` 对应 Java 的 `StringBuilder.append`。

立即练习

依次改写 `course-schedule.cxx`、`coin-change.cxx`、`binary-tree-maximum-path-sum.cxx`。每题先写注释：状态是什么？不变量是什么？Java 容器为什么是这个？

Part III

刷题落地

Chapter 6

Java 刷题 workflow：写得快，也要写得稳

6.1 LeetCode 模板

```
1 import java.util.*;
2
3 class Solution {
4     public int solve(int[] nums) {
5         // 1. 定义不变量
6         // 2. 处理空输入和边界
7         // 3. 写主循环
8         return 0;
9     }
10 }
```

如果题目提供 `ListNode`、`TreeNode`、`Node`，不要重复定义。提交前删除 `main`、调试输出和本地 `helper` 的冲突定义。

6.2 从你代码库抽出的题型清单

你的 C++ 文件	核心模式	Java 首选工具
3sum.cxx、 container-with-most-water.cxx	排序 + 双指针	Arrays.sort、数组下标
course-schedule.cxx、 number-of-islands.cxx	图搜索、拓扑排序	ArrayList、ArrayDeque、boolean[]
merge-k-sorted-lists.cxx find-median-from-data-stream.cxx	堆	PriorityQueue + Comparator
coin-change.cxx、 edit-distance.cxx、 LCS	DP	int[]、int[][]

你的 C++ 文件	核心模式	Java 首选工具
lru-cache.cxx、 time-based-key-value-store.cxx	设计题	HashMap、LinkedHashMap、二分
fhq-treap.cxx、 kruskal.cxx	高级数据结构	内部类、record、long
regular-expression-match.cxx、 shortest-palindrome.cxx	字符串/DP	char[]、StringBuilder

6.3 提交前 90 秒检查

1. **空输入**：null、空数组、单节点、容量 0 是否处理？
2. **整数**：路径和、乘积、平均数是否需要 long？
3. **Comparator**：是否用了 Integer.compare 而不是相减？
4. **堆方向**：Java 默认小顶堆；大顶堆是否传了 Comparator.reverseOrder()？
5. **队列**：是否使用 ArrayDeque，且没有放 null？
6. **状态重置**：成员变量 answer、visited 是否在 public 方法开头重置？

6.4 性能不降级的三个动作

1. 频繁索引的数值状态用 int[]，不要无脑 List<Integer>。
2. 字符串构造用 StringBuilder。
3. 需要容量时预分配：new ArrayList<>(expectedSize)，但不要为了微优化牺牲可读性。

递归深度

Java 栈通常比 C++ 算法环境更容易在极深 DFS 时溢出。普通二叉树题可递归；链状图、十万节点树、number-of-islands 的极端输入，更安全的是显式 ArrayDeque。

Chapter 7

速查表与最后冲刺

7.1 最常用 Java 片段

```
1 // 邻接表
2 List<List<Integer>> graph = new ArrayList<>();
3 for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
4
5 // 访问标记
6 boolean[] visited = new boolean[n];
7
8 // BFS
9 Deque<Integer> queue = new ArrayDeque<>();
10 queue.offer(start);
11 while (!queue.isEmpty()) {
12     int current = queue.poll();
13 }
14
15 // 最小堆 / 最大堆
16 PriorityQueue<Integer> minHeap = new PriorityQueue<>();
17 PriorityQueue<Integer> maxHeap =
18     new PriorityQueue<>(Comparator.reverseOrder());
19
20 // 哈希表默认容器
21 Map<Integer, List<Integer>> groups = new HashMap<>();
22 groups.computeIfAbsent(key, ignored -> new ArrayList<>()).add(value);
23
24 // 排序
25 Arrays.sort(nums);
26 Arrays.sort(points, (a, b) -> Integer.compare(a[0], b[0]));
```

7.2 C++ 到 Java 的高频替换

当你想写 C++	Java 中先想
nullptr	null, 并先决定在哪里检查
new Node(...) + delete	new Node(...); 只维护可达关系
vector<int> dp	int[] dp
unordered_map[key]	getOrDefault 或 computeIfAbsent
priority_queue<int>	PriorityQueue<Integer> (最小堆)
greater<int>	Comparator.reverseOrder() (用于最大堆)
sort(v.begin(),v.end())	Arrays.sort(array) 或 list.sort(...)
INT_MIN	Integer.MIN_VALUE
long long	long

7.3 你应该保留的 C++ 优点

- 写代码前先说清楚复杂度和不变量;
- 图、树、堆题先定义状态, 再动手;
- 对边界、空指针、哨兵和溢出保持敏感;
- 手写 LRU、Treap 的经历保留了数据结构直觉, 不要因为 Java 有库就丢掉。

毕业标准

当你看到“图 + 依赖关系”时, 第一反应是拓扑排序; 看到“流数据中位数”时, 第一反应是双堆; 看到 Java 时, 第一反应也应当是相同的算法, 只是顺手拿起 ArrayDeque、HashMap、PriorityQueue 和 StringBuilder。

最终模拟

限时 45 分钟, 用 Java 完成以下三题: 课程表、LRU Cache、数据流中位数。完成后逐项执行第六章的 90 秒检查。若你卡住的是 API, 就重做本章速查; 若你卡住的是状态设计, 回到对应 C++ 解法先读不变量, 不要急着看 Java 答案。

附录 A

Leet C++ 源码全量索引

本书参考的目录包含 47 份 C++ 解法和一个 Treap 头文件。下面的索引不是题目答案，而是把每份现有代码接到 Java 训练动作上；按这张表逐题改写，能够覆盖你的全部既有习惯。

源文件	主要 C++ 习惯	Java 迁移重点
3sum.cxx	sort + 双指针	Arrays.sort、数组下标
3sum-closest.cxx	双指针、距离比较	long 与 Math.abs
airbnb-pagination-dedup.cxx	分页、去重	LinkedHashSet、状态封装
best-time-to-buy-and-sell-stock.cxx	单遍扫描	Math.min/max
binary-tree-maximum-path-sum.cxx	DFS + 引用答案	成员变量/递归返回值
clone-graph.cxx	Node*、BFS	Map<Node,Node>、 ArrayDeque
coin-change.cxx	vector DP	int[]、Arrays.fill
combination-sum.cxx	回溯	ArrayList 路径复制
container-with-most-water.cxx	双指针	数组与边界
course-schedule.cxx	手写邻接链表	List<List<Integer>>、拓 扑排序
edit-distance.cxx	二维 DP	int[][]
fhq-treap.cxx / .h	裸节点、split/merge	内部类、递归不变量
find-first-and-last-position*.cxx	二分边界	半开区间模板
find-median-from-data-stream.cxx	双 priority_queue	PriorityQueue 堆方向
flatten-nested-list-iterator.cxx	迭代器	Iterator<Integer>、栈
generate-parentheses.cxx	DFS 构造字符串	StringBuilder 回溯
house-robber.cxx	线性 DP	滚动变量
integer-to-roman.cxx	映射与拼接	数组表、StringBuilder
kruskal.cxx	并查集	int[] parent/rank
letter-combinations*.cxx	递归组合	List<String>
longest-common-subsequence.cxx	二维 DP	数组初始化
longest-happy-prefix.cxx	KMP 前缀函数	char[]
longest-increasing-subsequence.cxx	patience sorting	Arrays.binarySearch 或 手写二分
longest-palindromic-substring.cxx	中心扩展/DP	字符串索引
lru-cache.cxx	手写双向链表	LinkedHashMap 或节点引 用

源文件	主要 C++ 习惯	Java 迁移重点
maximal-rectangle.cxx	单调栈	ArrayDeque<Integer>
merge-k-sorted-lists.cxx	堆 + 节点	PriorityQueue<ListNode>
minimum-path-sum.cxx	网格 DP	原地/一维 DP
next-permutation.cxx	原地数组操作	swap helper
number-of-islands.cxx	DFS/BFS 网格	boolean[][]、显式队列
pour-water.cxx	模拟	清晰的状态机循环
regular-expression-matching.cxx	递归/DP	memo key 或二维表
remove-nth-node-from-end.cxx	快慢指针	dummy node 引用
reverse-integer.cxx	数字溢出	long 中间值
reverse-nodes-in-k-group.cxx	链表分组	引用重连顺序
shortest-palindrome.cxx	KMP	StringBuilder.reverse()
sliding-puzzle.cxx	BFS 状态编码	HashSet<String>
string-to-integer-atoi.cxx	解析/溢出	边界与 long
substring-with-concatenation*.cxx	滑窗 + 哈希	HashMap 计数
swap-nodes-in-pairs.cxx	链表指针	dummy node
test.cxx	本地测试	JUnit/简单 main
text-justification.cxx	字符串布局	StringBuilder
time-based-key-value-store.cxx	map + upper bound	HashMap + 手写二分
Trapping-Rain-Water.cxx	双指针/单调栈	int[]、Deque
valid-sudoku.cxx	集合/位标记	boolean[] 或 bit mask
zigzag-conversion.cxx	多行字符收集	StringBuilder[]

推荐改写顺序

先做股票、硬币兑换、课程表、图克隆、数据流中位数和 LRU。这六题分别覆盖 Java 数组、DP、Collections、对象引用、Comparator 和设计题；其余题会自然变成同一种语言。

附录 B

你的 Java 迁移题单

下面是本次指定的完整题单。不要按 LeetCode 编号顺序硬刷；按 Java 需要形成的新习惯分组，才能把 C++ 的算法能力快速迁移过来。

B.1 第一组：数组、双指针、排序与基础 API

- 1 两数之和 (简单): HashMap 的第一题；建立“查补数再插当前值”的默认动作。
- 11 盛最多水的容器 (中等)、15 三数之和 (中等)、16 最接近的三数之和 (中等)、42 接雨水 (困难): 数组下标与双指针。
- 31 下一个排列 (中等)、34 查找第一个和最后一个位置 (中等)、48 旋转图像 (中等): 原地数组修改、二分边界与矩阵坐标。
- 121 买卖股票的最佳时机 (简单): Math.min/Math.max 和单遍状态。
- 36 有效的数独 (中等): boolean[]、bit mask 或 HashSet 的取舍。

B.2 第二组：对象引用、链表与设计题

- 2 两数相加 (中等)、19 删除倒数第 N 个结点 (中等)、24 两两交换节点 (中等)、25 K 个一组翻转链表 (困难): ListNode、dummy node 和引用重连。
- 23 合并 K 个升序链表 (困难): PriorityQueue<ListNode> 与 Comparator。
- 133 克隆图 (中等): Map<Node, Node>, 用对象身份而非节点值做 key。
- 146 LRU 缓存 (中等)、460 LFU 缓存 (困难)、981 基于时间的键值存储 (中等): HashMap、LinkedHashMap、双向链表和手写二分。
- 341 扁平化嵌套列表迭代器 (中等): Iterator 与 ArrayDeque。

B.3 第三组：字符串、栈和构造

- 3 无重复字符的最长子串 (中等)、30 串联所有单词 (困难): 滑动窗口与字符/单词计数。
- 5 最长回文子串 (中等)、214 最短回文串 (困难)、336 回文对 (困难)、1392 最长快乐前缀 (困难): 中心扩展、KMP、哈希与 char[]。
- 6 Z 字形变换 (中等)、12 整数转罗马 (中等)、17 电话号码组合 (中等)、22 括号生成 (中等)、68 文本左右对齐 (困难): StringBuilder 与回溯。
- 7 整数反转 (中等)、8 atoi (中等): long 中间值与溢出边界。
- 10 正则表达式匹配 (困难)、32 最长有效括号 (困难)、459 重复的子字符串 (简单): DP、栈和字符串周期。

B.4 第四组：DP、树、图、堆与最短路

- 39 组合总和 (中等)、64 最小路径和 (中等)、72 编辑距离 (中等)、85 最大矩形 (困难)、198 打家劫舍 (中等)、300 最长递增子序列 (中等)、322 零钱兑换 (中等)、1143 最长公共子序列 (中等): int[]、int[][] 与循环方向。
- 124 二叉树最大路径和 (困难): 递归返回值与成员答案字段。
- 127 单词接龙 (困难)、200 岛屿数量 (中等)、207 课程表 (中等)、695 岛屿最大面积 (中等)、733 图像渲染 (简单)、743 网络延迟时间 (中等)、773 滑动谜题 (困难): ArrayDeque、PriorityQueue、boolean[]。
- 295 数据流的中位数 (困难): 一大一小两个 PriorityQueue; 记住 Java 默认是小顶堆。

推荐执行法

每天从每组各选一题，共三题。先在 ~/leet-java 中用 ./scripts/new-problem.sh <slug> <array|list|tree> 建目录，再把 Solution.java 的最终版本粘贴到 LeetCode。每题完成后记录：你卡的是算法、Java API、还是对象/泛型语义。只要连续复盘两周，Java 会从“翻译 C++”变成自然语言。